CSC14120 – PARALLEL PROGRAMMING

FINAL PROJECT

Midterm Submission: Proposal and Sequential Version

Due: end of week 7 (23:59)

Weight: 10% of the project grade. The remaining 90% comes from the final submission and oral defense.

1. Purpose

The midterm checks that each team has a working, measured sequential baseline and a clear plan before starting the CUDA work. It is graded with a simple checklist; the real effort of the project belongs to the final submission.

2. What to submit

Submit one zip file named GroupXX_Midterm.zip through the course website. Only one member submits for the team. The zip contains:

```
GroupXX_Midterm.zip
├── GroupXX_Midterm.ipynb      report (Colab notebook, all outputs saved)
└── src/                       source code
    ├── common.h, cuda_utils.cuh, versions.h, main.cu, stb_impl.cpp
    ├── v0_<name>.cpp          your sequential version
    └── ...                    any other .c / .cpp / .h files you added
```

- **Report (.ipynb):** contains all the content in section 3: explanations, results, timing tables, charts, and panoramas. It does not need to show the full source code; include only short excerpts where they help explain a design choice.
- **Code (src/):** all C/C++/CUDA source files needed to build the program, as separate files (.c, .cpp, .cu, .h, .cuh). Add a comment at the top of every file you wrote or modified, listing the authors and any external source it was adapted from.
- **Dataset:** a link in the notebook to the team's dataset (e.g., a Google Drive zip with the raw photos and the prepared dataset/ folder).

3. Required content

3.1 Team information and work plan

Member names and student IDs, and a table showing who will be responsible for which stages and versions until the final submission.

3.2 Problem understanding

A explanation of the stitching pipeline in the team's own words.

3.3 Chosen methods

The method used for each of the six stages, the main parameters (e.g., number of keypoints, ratio-test threshold, RANSAC iterations), and a brief justification based on both output quality and suitability for the GPU.

3.4 Sequential version (V0)

- Single-threaded C/C++ that runs the complete pipeline, using the interfaces in the starter notebook.
- The code may be the team's own or adapted from open-source code. Any reused code must be cited in the source file and in the notebook.
- No multithreading (OpenMP, `std::thread`) and no hand-written SIMD, so that V0 is a fair baseline.

3.5 V0 results and profiling

- **Correctness:** the homography error on the synthetic pair, compared against the ground truth.
- **Panoramas:** one output per site from the team's dataset.
- **Timing:** a per-stage timing table for all three resolutions (small, medium, large).
- **Hot stages:** which stages take the most time, and how this changes with resolution.
- **Amdahl estimate:** if the stages you plan to accelerate take a fraction p of V0's time, the end-to-end speedup can never exceed $1 / (1 - p)$, however fast the GPU part becomes. For example, if the chosen stages take 90% of the time, the maximum speedup is 10×. Report p and this limit for your plan.

3.6 Dataset

The registered sites and prepared image pairs, meeting the requirements in the Datasets section of the project spec (at least 3 sites, 1 standard pair per site, 1 challenging pair).

3.7 GPU plan

- Which stages the team will move to the GPU, and why, based on the profiling in 3.5.
- For each of those stages, the planned thread mapping for V1 (what one thread, warp, or block computes).
- Two or three optimization ideas the team expects to try in later versions, and the main risks it foresees.

4. Requirements

- **One source of truth for code.** The notebook must build the program from the submitted src/ files, not from separate copies written inside the notebook. The setup cell copies the src/ folder into the Colab session (by uploading it or from the team's shared Drive folder) and compiles it.
- **Reproducible.** The notebook runs from top to bottom with Runtime → Run all on a fresh Colab session, once the src/ folder is available.
- **Numbers from outputs.** Every number in the text comes from an output cell in the notebook.